

MAR ATHANASIOS COLLEGE FOR ADVANCED STUDIES, TIRUVALLA
(MACFAST)

(Affiliated to Mahatma Gandhi University, Kottayam)



MACFAST (AUTONOMOUS)

MCACP207 DATA STRUCTURES LAB USING C

Department of Computer Applications

Mar Athanasios College for Advanced Studies, Tiruvalla

(MACFAST) AUTONOMOUS

Kerala, India, Pin-689101

www.macfast.org

www.macfast.ac.in

MAR ATHANASIOS COLLEGE FOR ADVANCED STUDIES, TIRUVALLA
(MACFAST)

(Affiliated to Mahatma Gandhi University, Kottayam)



MACFAST (AUTONOMOUS)

MCACP207 DATA STRUCTURES LAB USING C

Name : PRINTO MATHEW
Register No : 253242210258
Semester : 2
Subject : DATA STRUCTURES LAB USING C

*Certified that this is a bonafide record of the practical work done by
Mr. PRINTO MATHEW Reg.No: 253242210258 in the Department of Computer Applications
during 2025-2027 at Mar Athanasios College for Advanced Studies, Tiruvalla ((MACFAST)
Autonomous).*

Faculty in Charge

Head of the Department

Examiners

1.....

2.....

INDEX

SL.NO	DATE	NAME OF PROGRAM	PAGE NO
1	18/02/2026	Linear Search	5
2	18/02/2026	Binary Search	9
3	26/02/2026	Polynomial Addition With 1-Dimensional Array	13
4	05/03/2026	Stack Operations	19
5	12/03/2026	Infix Expression to Postfix Expression Using Stack	27
6	12/03/2026	Evaluate Postfix Expression Using Stack	37
7	16/03/2026	Queue Operations	43
8	23/03/2026	Circular Queue Operations Using Array	49
9	23/03/2026	Insertion Sort	57
10	07/05/2026	Merge Sort	61
11	19/05/2026	Quick Sort	67
12	20/05/2026	Singly Linked List Operations	73
13	21/05/2026	Implement Binary Tree & Binary Search Tree	87

OUTPUT

```
----- Linear Search -----  
Enter array size:          5  
Enter the 5 elements:  
10  
41  
27  
18  
3  
Enter the element to search:    27  
27 found at position 3
```

```
----- Linear Search -----  
Enter array size:          5  
Enter the 5 elements:  
18  
5  
91  
51  
25  
Enter the element to search:    10  
Element does not found..
```

PROGRAM – 1

AIM : Program to search an element in an array using linear search.

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,a[50],key,n,flag=0;
    clrscr();
    printf("----- Linear Search -----\\n");
    printf("Enter array size:\\t");
    scanf("%d",&n);
    printf("Enter the %d elements:\\n",n);
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("Enter the element to search:\\t");
    scanf("%d",&key);
    for(i=0;i<n;i++)
    {
        if(key == a[i])
        {
            flag =1;
            break;
        }
    }

    if(flag == 1)
    {
        printf("%d found at position %d",key,i+1);
    }
}
```



```
else{  
    printf("Element does not found..");  
}  
getch();  
}
```

OUTPUT

```
----- Binary Search -----  
Enter the size of array:  
5  
Enter the elements in ascending order:  
23  
54  
67  
81  
98  
Enter the element to be searched:      12  
Element not found.._
```

```
----- Binary Search -----  
Enter the size of array:  
5  
Enter the elements in ascending order:  
23  
54  
67  
81  
98  
Enter the element to be searched:      81  
81 found at postion 4
```

PROGRAM – 2

AIM : Program to search an element in an array using binary search.

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[50],n,i,key,flag=0,mid,high,low=0;
    clrscr();
    printf("----- Binary Search -----\\n");
    printf("Enter the size of array:\\n");
    scanf("%d",&n);
    printf("Enter the elements in ascending order:\\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }
    printf("Enter the element to be searched:\\t");
    scanf("%d",&key);
    high = n - 1;
    while(low <= high)
    {
        mid = (low + high)/2;
        if(a[mid] == key)
        {
            flag =1;
            break;
        }
        else if(key < a[mid])
        {
            high = mid - 1;
        }
    }
}
```



```
    }  
    else{  
        low = mid + 1;  
    }  
}  
if(flag == 1)  
{  
    printf("%d found at postion %d",key,mid+1);  
}  
else{  
    printf("Element not found..");  
}  
getch();  
}
```

OUTPUT

```
Enter the no of factors in 1st polynomial
2
Enter the no of factors in the 2nd polynomial
2
Enter the exponent and coefficent of 1st polynomial
6
3
3
4
Enter the exponent and coefficent of 2nd polynomial
2
5
1
3
The resultant polynomial is:
      4      6      3      3      4      2      5      1      3
```

PROGRAM – 3

AIM : Program to perform polynomial addition with 1 dimensional array.

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[20], b[20], c[20], p, q, r, m, n, i;
    p = q = r = 2;
    clrscr();
    printf("Enter the no of factors in 1st polynomial\n");
    scanf("%d", &m);
    printf("Enter the no of factors in the 2nd polynomial\n");
    scanf("%d", &n);
    a[1] = m;
    b[1] = n;
    printf("Enter the exponent and coefficient of 1st polynomial\n");
    for(i = 2; i <= (2 * m) + 1; i++)
    {
        scanf("%d", &a[i]);
    }
    printf("Enter the exponent and coefficient of 2nd polynomial\n");
    for(i = 2; i <= (2 * n) + 1; i++)
    {
        scanf("%d", &b[i]);
    }
    while((p <= 2 * m) && (q <= 2 * n))
    {
```



```

if(a[p] == b[q])
{
    c[r + 1] = a[p + 1] + b[q + 1];

    if(c[r + 1] != 0)
    {
        c[r] = a[p];
        r = r + 2;
    }
    p = p + 2;
    q = q + 2;
}
else if(a[p] < b[q])
{
    c[r + 1] = b[q + 1];
    c[r] = b[q];
    q = q + 2;
    r = r + 2;
}

else
{
    c[r + 1] = a[p + 1];
    c[r] = a[p];
    p = p + 2;
    r = r + 2;
}
}

```



```

while(p <= (2 * m))
{
    c[r] = a[p];
    c[r + 1] = a[p + 1];
    p = p + 2;
    r = r + 2;
}
while(q <= (2 * n))
{
    c[r] = b[q];
    c[r + 1] = b[q + 1];
    q = q + 2;
    r = r + 2;
}
c[1] = r / 2 - 1;
printf("The resultant polynomial is:\n");
for(i = 1; i < r; i++)
{
    printf("\t %d", c[i]);
}
getch();
}

```

OUTPUT

```
---- Stack Operations ----
```

1. PUSH
2. POP
3. PEEK
4. DISPLAY ALL
5. EXIT

```
-----
```

Enter choice

1

Enter element to be inserted:

10

Stack Push

```
---- Stack Operations ----
```

1. PUSH
2. POP
3. PEEK
4. DISPLAY ALL
5. EXIT

```
-----
```

Enter choice

1

Stack Overflow

Stack Overflow

PROGRAM – 4

AIM : Program to perform linear stack operations.

SOURCE CODE:

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

#define MAX 5

int top = -1;
int stack[MAX];

void push(){
    int value;
    if(top == MAX -1)
    {
        printf("Stack Overflow \n");
    }
    else{
        printf("Enter element to be inserted: \n");
        scanf("%d",&value);
        top++;
        stack[top]= value;
    }
}
```

OUTPUT

```
---- Stack Operations ----  
1. PUSH  
2. POP  
3. PEEK  
4. DISPLAY ALL  
5. EXIT  
-----  
Enter choice  
4  
Stack Elements are:  
63      23      12      11      10
```

Display All Stack Elements

```
---- Stack Operations ----  
1. PUSH  
2. POP  
3. PEEK  
4. DISPLAY ALL  
5. EXIT  
-----  
Enter choice  
3  
Top of stack: 63
```

Peek

```
void pop(){
    if(top == -1)
    {
        printf("Stack Underflow \n");
    }
    else{
        printf("Element deleted: %d\n",stack[top]);
        top--;
    }
}
```

```
void peek(){
    if(top == -1)
    {
        printf("Stack is empty\n");
    }
    else{
        printf("Top of stack: %d\n",stack[top]);
    }
}
```

```
void display(){
    int i;
    if(top == -1)
    {
        printf("Stack is empty\n");
    }
}
```

OUTPUT

```
---- Stack Operations ----  
1. PUSH  
2. POP  
3. PEEK  
4. DISPLAY ALL  
5. EXIT  
-----  
Enter choice  
2  
Element deleted: 63
```

Stack Pop

```

else{
    printf("Stack Elements are:\n");
    for(i=top;i>=0;i--)
    {
        printf("%d\t",stack[i]);
    }
}
}

void main()
{
    int choice;
    clrscr();
    while(1)
    {
        printf("---- Stack Operations ----\n");
        printf("1. PUSH\n");
        printf("2. POP\n");
        printf("3. PEEK\n");
        printf("4. DISPLAY ALL\n");
        printf("5. EXIT\n");
        printf("Enter choice\n");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1:    push();
                      break;
            case 2:    pop();
                      break;

```



```
case 3: peek();
        break;
case 4: display();
        break;
case 5: exit(0);
default: printf("!! Invalid input !!");
}
}
getch();
}
```

OUTPUT

```
Enter the expression
(A*B)+D/C
POSTFIX EXPRESSION IS
AB*DC/+_
```

PROGRAM – 5

AIM : Program to Convert an infix expression to a postfix expression using stack (Array).

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
char st[15];
int top=-1;
int prio(char c);
void push(char st[],char);
char pop(char st[]);
void postfix(char infix[], char posx[]);

char infix[20];
char posx[20];
char temp;
void main()
{
    clrscr();
    printf("Enter the expression\n");
    gets(infix);
    push(st,'#');
    postfix(infix,posx);
    printf("POSTFIX EXPRESSION IS\n");
    printf("%s", posx);
    getch();
}
```



```

void postfix(char infix[20], char posx[20])
{
    int i=0,j=0;

    while(infix[i]!='\0')
    {
        if(infix[i]=='(')
        {
            push(st,infix[i]);
        }

        else if(infix[i]==')')
        {
            temp=pop(st);

            while(temp!='(')
            {
                posx[j]=temp;
                j++;

                temp=pop(st);
            }
        }
        else if((infix[i]>='A' && infix[i]<='Z') || (infix[i]>='a' && infix[i]<='z'))
        {
            posx[j]=infix[i];
            j++;
        }
    }
}

```



```

else
{
    while((prio(st[top])) >= (prio(infix[i])))
    {
        posx[j]=pop(st);
        j++;
    }
    push(st,infix[i]);
}
i++;
}
while(top>0)
{
    posx[j]=pop(st);
    j++;
}
posx[j]='\0';
}

```

```

int prio(char c)
{
    if(c=='^')
    {
        return 5;
    }
    else if((c=='/') || (c=='*'))
    {
        return 4;
    }
}

```



```

else if((c=='+') || (c=='-'))
{
    return 3;
}
else
{
    return 2;
}
}

void push(char st[], char ch)
{
    if(top==14)
    {
        printf("\nOVERFLOW");
    }

    else
    {
        top=top+1;
        st[top]=ch;
    }
}

char pop(char st[])
{
    char b;

    if(top==-1)
    {

```



```
    printf("\nUNDERFLOW");  
}  
  
else  
{  
    b=st[top];  
    top--;  
}  
  
return b;  
}
```

OUTPUT

```
Enter the expression :: 82/7+
```

```
The result of expression 82/7+ = 11_
```

PROGRAM – 6

AIM : Program to evaluate a postfix expression using stack (Array).

SOURCE CODE:

```
#include <stdio.h>

#include <conio.h>

#include <ctype.h>

int stack[20];

int top = -1;

void push(int x)

{

    stack[++top] = x;

}

int pop()

{

    return stack[top--];

}

void main()

{

    char exp[20];

    char *e;

    int n1, n2, n3, num;

    clrscr();

    printf("Enter the expression :: ");

    scanf("%s", exp);
```



```

e = exp;

while (*e != '\0')
{
    if (isdigit(*e))
    {
        num = *e - 48; // convert char to int

        push(num);
    }
    else
    {
        n1 = pop();
        n2 = pop();
        switch (*e)
        {
            case '+':
                n3 = n1 + n2;

                break;

            case '-':
                n3 = n2 - n1;

                break;

            case '*':
                n3 = n1 * n2;

                break;

```



```
        case '/':  
            n3 = n2 / n1;  
            break;  
        }  
        push(n3);  
    }  
    e++;  
}  
printf("\nThe result of expression %s = %d", exp, pop());  
getch();  
}
```

OUTPUT

```
---- Queue Operation ----  
  1.Enqueue  
  2.Dequeue  
  3.Display  
  4.Exit  
Enter your choice:  
1  
Insert item into Queue  
10
```

Enqueue

```
---- Queue Operation ----  
  1.Enqueue  
  2.Dequeue  
  3.Display  
  4.Exit  
Enter your choice:  
1  
Queue overflow
```

Queue Underflow

PROGRAM – 7

AIM : Program to perform linear queue operations using array.

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
#include<stdlib.h>
#define MAX 5

int queue[MAX];
int front = -1;
int rear = -1;

void enqueue()
{
    int value;
    if(rear == MAX - 1)
    {
        printf("Queue overflow\n");
    }
    else
    {
        if(front == -1)
            front = 0;
        printf("Insert item into Queue\n");
        scanf("%d",&value);
        rear++;
        queue[rear]= value;
    }
}
```

OUTPUT

```
---- Queue Operation ----  
1.Enqueue  
2.Dequeue  
3.Display  
4.Exit  
Enter your choice:  
3  
Queue Elements:  
10      11      22      71
```

Display all elements in queue

```
---- Queue Operation ----  
1.Enqueue  
2.Dequeue  
3.Display  
4.Exit  
Enter your choice:  
2  
Element deleted: 10
```

Dequeue

```
}
```

```
void dequeue()
```

```
{
```

```
if(front == -1 || front > rear)
```

```
{
```

```
    printf("Queue Underflow \n");
```

```
}
```

```
else
```

```
{
```

```
    printf("Element deleted: %d\n",queue[front]);
```

```
    front++;
```

```
}
```

```
}
```

```
void display()
```

```
{
```

```
int i;
```

```
if(front == -1 || front > rear)
```

```
{
```

```
    printf("Queue is empty");
```

```
}
```

```
else
```

```
{
```

```
    for(i=front;i<rear;i++)
```

```
{
```

```
    printf("%d\t",queue[i]);
```

```
}
```

```
printf("\n");
```



```

}
}

void main()
{
    int choice;
    clrscr();
    printf("---- Queue Operation ----\n 1.Enqueue\n2.Dequeue\n3.Display\n4.Exit\n");
    while(1)
    {
        printf("Enter your choice:\n");
        scanf("%d",&choice);
        switch(choice)
        {
            case 1: enqueue();
                    break;
            case 2: dequeue();
                    break;
            case 3: display();
                    break;
            case 4: exit(0);
                    break;
            default: printf("!! Invalid Input !!");
        }
    }
    getch();
}

```

OUTPUT

```
---- CIRCULAR QUEUE ----
```

- 1. ENQUEUE
- 2. DEQUEUE
- 3. DISPLAY ALL
- 4. EXIT

Enter Your Choice:

1

Enter the element: 5

Enqueue

- 1. ENQUEUE
- 2. DEQUEUE
- 3. DISPLAY ALL
- 4. EXIT

Enter Your Choice:

1

Queue Overflow

Queue Overflow

PROGRAM – 8

AIM : Program to perform circular queue operations using array.

SOURCE CODE:

```
#include<stdio.h>

#include<conio.h>

#include<stdlib.h>

#define MAX 5


int queue[MAX];

int front = -1;

int rear = -1;

void enqueue()

{

    int value;

    if(front == -1 && rear == -1)

    {

        front = 0;

        rear = 0;

        printf("Enter the element:\t");

        scanf("%d",&value);

        queue[rear] = value;

    }

}
```

OUTPUT

```
1. ENQUEUE
2.DEQUEUE
3.DISPLAY ALL
4. EXIT
Enter Your Choice:
3
Elements of Circular Queue:
5 10 15 20 25
```

Display All Queue Elements

```
1. ENQUEUE
2.DEQUEUE
3.DISPLAY ALL
4. EXIT
Enter Your Choice:
2
Deleted element is: 5
```

Dequeue

```

else if((rear+1)%MAX ==front)
{
    printf("Queue Overflow\n");
}
else
{
    rear = (rear +1) % MAX;
    printf("Enter the element to insert:\t");
    scanf("%d",&value);
    queue[rear] = value;
}
}

void dequeue()
{
    if(front == -1 && rear == -1)
    {
        printf("Queue Underflow \n");
    }
    else if( front == rear)
    {
        printf("Deleted element is: %d\n",queue[front]);
        front = -1;
        rear = -1;
    }
}

```



```

else

{

    printf("Deleted element is: %d\n",queue[front]);

    front = (front + 1 ) % MAX;

}

}

void display()

{

    int i= front;

    if(front == -1 && rear == -1)

    {

        printf("Queue is empty\n");

    }

    else

    {

        printf("Elements of Circular Queue:\n");

        printf("%d ",queue[i]);

        while(i != rear)

        {

            i = (i+1)% MAX;

            printf("%d ",queue[i]);

        }

    }

}

```



```

void main()

{

int choice;

clrscr();

printf("---- CIRCULAR QUEUE ----\n");

while(1)

{

printf("\n1. ENQUEUE \n 2.DEQUEUE \n 3.DISPLAY ALL\n 4. EXIT \n");

printf("Enter Your Choice: \n");

scanf("%d",&choice);

switch(choice)

{

case 1: enqueue();

break;

case 2: dequeue();

break;

case 3: display();

break;

case 4: exit(0);

default: printf("INVAALID INPUT.....\n");

}

}

getch();

}

```

OUTPUT

```
---- Insertion Sort ----  
Enter no of elements:  5  
Enter elements:  
2  
11  
8  
1  
6  
Before Sorting:  
2      11      8      1      6  
After Sorting:  
1      2      6      8      11      _
```

PROGRAM – 9

AIM : Program to perform insertion sort.

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>

void main()
{
    int a[100];
    int i,j,n,key;
    clrscr();
    printf("---- Insertion Sort ----\n");
    printf("Enter no of elements: \t");
    scanf("%d",&n);
    printf("Enter elements:\n");
    for(i=0;i<n;i++)
    {
        scanf("%d",&a[i]);
    }

    printf("Before Sorting:\n");
    for(i=0;i<n;i++)
    {
        printf("%d\t",a[i]);
    }
    printf("\n");
```



```

//
for(i=1;i<n;i++)
{
    key= a[i];
    j= i-1;

    while(j>=0 && a[j] > key)
    {
        a[j+1] = a[j];
        j= j - 1;
    }
    a[j+1] = key;
}

printf("After Sorting:\n");
for(i=0;i<n;i++)
{
    printf("%d\t",a[i]);
}

getch();
}

```

OUTPUT

```
---- Merge Sort ----  
Enter number of elements: 5  
Enter elements:  
19  
8  
15  
21  
2  
Before sorting:  
19      8      15      21      2  
Sorted array:  
2      8      15      19      21      _
```

PROGRAM – 10

AIM : Program to perform merge sort.

SOURCE CODE:

```
#include<stdio.h>
#include<conio.h>
// Function to merge two halves
void merge(int arr[], int l, int m, int r) {
    int i = l, j = m + 1, k = 0;
    int temp[100];

    while (i <= m && j <= r) {
        if (arr[i] < arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= m) {
        temp[k++] = arr[i++];
    }

    while (j <= r) {
        temp[k++] = arr[j++];
    }

    // Copy back to original array
    for (i = l, k = 0; i <= r; i++, k++) {
        arr[i] = temp[k];
    }
}
```



```

// Merge Sort function
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        mergeSort(arr, l, m);    // left half
        mergeSort(arr, m + 1, r); // right half
        merge(arr, l, m, r);     // merge both
    }
}

void main() {
    int arr[100], n, i;
    clrscr();
    printf("---- Merge Sort ----\n");
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    printf("Before sorting:\n");
    for(i=0;i<n;i++)
    {
        printf("%d\t",arr[i]);
    }
    printf("\n");
    mergeSort(arr, 0, n - 1);
}

```



```
printf("Sorted array:\n");  
for (i = 0; i < n; i++)  
{  
    printf("%d\t", arr[i]);  
}  
getch();  
}
```

OUTPUT

```
----- Quick Sort -----  
Enter number of elements:      5  
Enter elements:  
90  
7  
18  
3  
84  
Sorted elements:  
3 7 18 84 90 _
```

PROGRAM – 11

AIM : Program to perform quick sort.

SOURCE CODE:

```
#include<stdio.h>

#include<conio.h>

void quicksort(int a[], int low, int high)

{
    int i, j, pivot, temp;

    if(low < high)

    {
        pivot = a[low]; //1st element as pivot elmt

        i = low;

        j = high;

        while(i < j)

        {
            while(a[i] <= pivot && i < high)

            {
                i++;
            }

            while(a[j] > pivot)

            {
                j--;
            }
        }
    }
}
```



```

        if(i < j)
        {
            temp = a[i];

            a[i] = a[j];

            a[j] = temp;

        }
    }

    a[low] = a[j];

    a[j] = pivot;

    quicksort(a, low, j - 1);

    quicksort(a, j + 1, high);

}

}

void main()

{

    int a[10], n, i;

    clrscr();

    printf("----- Quick Sort -----\\n");

    printf("Enter number of elements:\\t");

    scanf("%d", &n);

    printf("Enter elements:\\n");

```



```
for(i = 0; i < n; i++)  
{  
    scanf("%d", &a[i]);  
}  
quicksort(a, 0, n - 1);  
printf("Sorted elements:\n");  
for(i = 0; i < n; i++)  
{  
    printf("%d ", a[i]);  
}  
getch();  
}
```

OUTPUT

```
Singly Linked List Operations
1. Insert at the End
2. Insert at the Beginning
3. Insert at a Specific Position
4. Delete from the Beginning
5. Delete from the End
6. Delete from a Specific Position
7. Display the List
8. Exit
Enter your choice: 1
Enter element: 10
Element 10 inserted at the end
```

Insert at the End

```
Singly Linked List Operations
1. Insert at the End
2. Insert at the Beginning
3. Insert at a Specific Position
4. Delete from the Beginning
5. Delete from the End
6. Delete from a Specific Position
7. Display the List
8. Exit
Enter your choice: 2
Enter element: 4
Element 4 inserted at the beginning
```

Insert at the Beginning

PROGRAM – 12

AIM : Program to create, add, delete and search in a singly linked list.

SOURCE CODE:

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

void insert_end(struct node **head, int data)
{
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = data;
    new_node->next = NULL;
    if (*head == NULL)
    {
        *head = new_node;
    }
    else
    {
        struct node *temp = *head;
        while (temp->next != NULL)
        {
            temp = temp->next;
        }
        temp->next = new_node;
    }
    printf("Element %d inserted at the end\n", data);
}
```

OUTPUT

```
Singly Linked List Operations
1. Insert at the End
2. Insert at the Beginning
3. Insert at a Specific Position
4. Delete from the Beginning
5. Delete from the End
6. Delete from a Specific Position
7. Display the List
8. Exit
Enter your choice:
                        3
Enter element: 14
Enter position: 8
Position out of range
```

Insert at a Specific Position

```
Singly Linked List Operations
1. Insert at the End
2. Insert at the Beginning
3. Insert at a Specific Position
4. Delete from the Beginning
5. Delete from the End
6. Delete from a Specific Position
7. Display the List
8. Exit
Enter your choice: 4
Element 4 deleted from the beginning
```

Delete from the Beginning

```

void insert_beginning(struct node **head, int data)
{
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = data;
    new_node->next = *head;
    *head = new_node;
    printf("Element %d inserted at the beginning\n", data);
}

```

```

void insert_at_position(struct node **head, int data, int position)
{
    struct node *temp=*head;
    int i;
    struct node *new_node = (struct node *)malloc(sizeof(struct node));
    new_node->data = data;
    if (position == 1)
    {
        new_node->next = *head;
        *head = new_node;
        printf("Element %d inserted at position %d\n", data, position);
        return;
    }
    for ( i = 1; temp != NULL && i < position - 1; i++)
    {
        temp = temp->next;
    }
    if (temp == NULL)
    {
        printf("Position out of range\n");
        free(new_node);
        return;
    }
}

```

OUTPUT

```
Singly Linked List Operations
1. Insert at the End
2. Insert at the Beginning
3. Insert at a Specific Position
4. Delete from the Beginning
5. Delete from the End
6. Delete from a Specific Position
7. Display the List
8. Exit
Enter your choice: 5
Element 10 deleted from the end
```

Delete from the End

```
Singly Linked List Operations
1. Insert at the End
2. Insert at the Beginning
3. Insert at a Specific Position
4. Delete from the Beginning
5. Delete from the End
6. Delete from a Specific Position
7. Display the List
8. Exit
Enter your choice: 6
Enter position: 2
Element 19 deleted from position 2
```

Delete from a Specific Position

```

new_node->next = temp->next;
temp->next = new_node;
printf("Element %d inserted at position %d\n", data, position);
}

void delete_beginning(struct node **head)
{
struct node *temp=*head;
    if (*head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    *head = (*head)->next;

    printf("Element %d deleted from the beginning\n", temp->data);
    free(temp);
}

void delete_end(struct node **head)
{
struct node *temp=*head;
    if (*head == NULL)
    {
        printf("List is empty\n");
        return;
    }

    if (temp->next == NULL)
    {
        printf("Element %d deleted from the end\n", temp->data);
        free(temp);
    }
}

```

OUTPUT

```
Singly Linked List Operations
1. Insert at the End
2. Insert at the Beginning
3. Insert at a Specific Position
4. Delete from the Beginning
5. Delete from the End
6. Delete from a Specific Position
7. Display the List
8. Exit
Enter your choice: 7
18 -> NULL
```

Display the list

```

        *head = NULL;
    return;
}
while (temp->next->next != NULL)
{
    temp = temp->next;
}
printf("Element %d deleted from the end\n", temp->next->data);
free(temp->next);
temp->next = NULL;
}

void delete_at_position(struct node **head, int position)
{
    struct node *temp;
    struct node *del;
    int i;
    temp=*head;
    if (*head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    if (position == 1)
    {
        *head = temp->next;
        printf("Element %d deleted from position %d\n", temp->data, position);
        free(temp);
        return;
    }

```



```

for ( i = 1; temp->next != NULL && i < position - 1; i++)
{
    temp = temp->next;
}

if (temp->next == NULL)
{
    printf("Position out of range\n");
    return;
}

del = temp->next;
temp->next = del->next;

printf("Element %d deleted from position %d\n", del->data, position);
free(del);
}

void display(struct node *head)
{
    struct node *temp=head;
    if (head == NULL)
    {
        printf("List is empty\n");
        return;
    }
    while (temp != NULL)
    {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }

    printf("NULL\n");
}

```



```

int main()
{
    struct node *head = NULL;
    int choice, value, pos;
    clrscr();
    do
    {
        printf("\nSingly Linked List Operations\n");
        printf("1. Insert at the End\n");
        printf("2. Insert at the Beginning\n");
        printf("3. Insert at a Specific Position\n");
        printf("4. Delete from the Beginning\n");
        printf("5. Delete from the End\n");
        printf("6. Delete from a Specific Position\n");
        printf("7. Display the List\n");
        printf("8. Exit\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch(choice)
        {
            case 1:
                printf("Enter element: ");
                scanf("%d", &value);
                insert_end(&head, value);
                break;
            case 2:
                printf("Enter element: ");
                scanf("%d", &value);
                insert_beginning(&head, value);
                break;

```



```

    case 3:
        printf("Enter element: ");
        scanf("%d", &value);
        printf("Enter position: ");
        scanf("%d", &pos);
        insert_at_position(&head, value, pos);
        break;
    case 4:
        delete_beginning(&head);
        break;
    case 5:
        delete_end(&head);
        break;
    case 6:
        printf("Enter position: ");
        scanf("%d", &pos);
        delete_at_position(&head, pos);
        break;
    case 7:
        display(head);
        break;
    case 8:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice\n");
}

}

while(choice != 8);
    return 0;
}

```

OUTPUT

```
Enter number of nodes: 4
Enter value: 34
Enter value: 31
Enter value: 91
Enter value: 7
```

```
Preorder Traversal:
34 -> 31 -> 7 -> 91 ->
```

```
Inorder Traversal:
7 -> 31 -> 34 -> 91 ->
```

```
Postorder Traversal:
7 -> 31 -> 91 -> 34 -> _
```

PROGRAM – 13

AIM : Program to implement the creation of binary tree and binary search tree and performing the traversals.

SOURCE CODE:

```
#include <stdio.h>
#include <conio.h>
#include <stdlib.h>

struct node {
int data;
struct node *left;
struct node *right;
};

struct node* create_node(int value) {
struct node *newnode = (struct node*) malloc(sizeof(struct node));
newnode->data = value;
newnode->left = NULL;
newnode->right = NULL;
return newnode;
}

struct node* insert(struct node *root, int value) {
if (root == NULL) {
return create_node(value);
}
if (value < root->data) {
root->left = insert(root->left, value);
}
else {
root->right = insert(root->right, value);
}
return root;
}
```



```

void preorder(struct node *root) {
if (root != NULL) {
printf("%d -> ", root->data);
preorder(root->left);
preorder(root->right);
}
}

```

```

void inorder(struct node *root) {
if (root != NULL) {
inorder(root->left);
printf("%d -> ", root->data);
inorder(root->right);
}
}

```

```

void postorder(struct node *root) {
if (root != NULL) {
postorder(root->left);
postorder(root->right);
printf("%d -> ", root->data);
}
}

```

```

void main() {
struct node *root = NULL;
int n, value, i;
printf("Enter number of nodes: ");
scanf("%d", &n);

for(i = 0; i < n; i++) {
printf("Enter value: ");
scanf("%d", &value);
root = insert(root, value);
}
}

```



```
printf("\nPreorder Traversal:\n");  
preorder(root);
```

```
printf("\n\nInorder Traversal:\n");  
inorder(root);
```

```
printf("\n\nPostorder Traversal:\n");  
postorder(root);  
getch();  
}
```